5 IMPLEMENTATION

We implemented two prototype versions of LLVM to study the impact of adapting the compiler to match the new semantics. First, we removed optimizations that are invalid in our semantics in order to create a version of LLVM that soundly implements its memory model. We confirmed this version fixes all the related bugs we have found. Second, we modified the "sound" compiler to regain some performance by adding optimizations that were not previously supported. We implemented our prototypes using LLVM 6.0.[6]

Making LLVM sound. LLVM propagated pointer equalities in several places; these had to be disabled. For example, InstSimplify (a peephole optimizer that does not create new instructions) transforms the IR equivalent of `(x == y) ? x : y` into `y`. In our semantics this transformation is correct for integers, but not for pointers, because it breaks data-flow provenance tracking. Similarly, we turned off GVN for pointer-typed variables.

A second kind of transformation that we had to disable were some integer-pointer conversions. LLVM (and GCC to some extent) treat a round-trip of pointer-to-integer and then integer-to-pointer casts as a no-op. This is incorrect under our semantics, and we removed this transformation—the IR equivalent of turning `(int*)(int)p` into `p`—in InstSimplify. Also related is a transformation from InstCombine (a peephole optimizer that potentially creates new instructions), which rewrites `(int)p == (int)q` into `p == q` for pointers `p` and `q`. This transformation was also removed, since comparing physical pointers is not equivalent to comparing logical pointers in our semantics.

A third kind of change is related to compiler-introduced type punning. There are optimizations in LLVM that transform, for example, load/store instructions of pointers into that of pointer-sized integers. This includes GVN when equivalent variables have different types, transforming a small `memcpy` into a pair of load/store instructions (e.g., where the intermediate value is an integer, and the target type is a pointer), etc. As we will discuss in Section 8, we had to disable all of these.

Finally, we disabled a few additional transformations that became invalid in our semantics including folding a pointer comparison against a stack-allocated object within a loop and a handful of peephole optimizations that changed pointer provenance. In total, we changed 419 lines of code.

Regaining performance. Making LLVM sound with respect to our memory model caused some small performance regressions in SPEC CPU 2017. We created a second prototype that attempts to regain that performance by adding optimizations that conform to our new semantics.

We changed how LLVM/Clang handles C/C++ pointer subtractions. The unmodified Clang lowers pointer subtraction into the IR equivalent of `(int)p - (int)q`. This is sound under our model, but since LLVM is very conservative when it encounters these kind of casts (it assumes the pointers escape), we unnecessarily lose precision in, e.g., alias analysis. The problem becomes even worse in our "sound" prototype. We added the **psub** instruction for pointer subtraction (described in Sec. 4.7) and modified Clang to use it. Since in our semantics **psub** does not escape if both operands are logical pointers, more aggressive optimization are now allowed.

We augmented alias analysis so that it becomes more precise when it encounters integer-to-pointer casts. Alias analysis can now conclude that a pointer cast from integer never aliases an unescaped object. Moreover, we also improved the analysis to assume that the pointer comparison instruction does not make its operands escape if they are both logical pointers.

Finally, we re-enabled GVN for pointer types for the following specific cases where it is guaranteed to be sound (for pointers `p` and `q` in the same equivalence class, where `p` is replaced with `q`):

- `q` is `NULL` or the result of an integer-to-pointer cast.
- `p` and `q` are logical pointers, and both are either dereferenceable or point to the same block.

[6] https://github.com/snu-sf/llvm-twin and https://github.com/snu-sf/clang-twin